



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN GENERAL DE PREGRADO

INGENIERÍA INFORMÁTICA

ASIGNATURA: INGENIERÍA DEL SOFTWARE II

SECCIÓN 1

“Liderazgo, Equipos y Cultura de Ingeniería”

DOCENTE:

Félix Márquez

ALUMNOS:

Luis Millán

V - 30.040.213

Endrys Flores

V - 30.451.556

Robert Castro

V - 30.994.049

Jesus Báez

V - 26.753.871

Ciudad Guayana, Septiembre del 2026.

ÍNDICE.

INTRODUCCIÓN	3
Pregunta 1.	4
1. Análisis del problema actual mediante la Ley de Conway.	4
1.2 ¿Por qué la estructura organizacional está garantizando el fracaso de la agilidad?	5
1.3 Rediseño Organizacional: Conformación de los Stream-Aligned Teams	6
STREAM-ALIGNED TEAM: CRÉDITOS	6
1.4 ¿Cómo quedarían conformados los equipos para soportar una futura migración a microservicios?	8
1.5 ¿Qué rol jugaría el antiguo "Gerente de Proyecto" en esta nueva estructura organizacional?	10
Pregunta 2.	10
2.1 Diagnóstico de la situación de revisión de código (Pull Requests)	11
2.2 Consecuencias a largo plazo en el rendimiento del equipo	12
2.3 Intervención, Reestructuración de la Revisión de Código y Retrospectiva	13
2.4 Estrategia de Intervención con el Arquitecto Principal	13
2.5 Reestructuración del Proceso de Revisión de Código	16
2.6 Dinámica para la Próxima Retrospectiva	18
CONCLUSIONES	21
REFERENCIAS	22

INTRODUCCIÓN

La gestión del talento humano, la estructura organizacional y las dinámicas de comunicación constituyen el núcleo crítico en la ingeniería de software moderna. Históricamente, las organizaciones han abordado el desarrollo de sistemas desde un paradigma mecanicista y taylorista, considerando a los profesionales como "recursos" intercambiables dentro de un diagrama de Gantt y reduciendo la gestión a un esquema de comando y control. Sin embargo, el desarrollo de software es una actividad intrínsecamente intensiva en conocimiento, altamente compleja y profundamente colaborativa, donde la estructura social y comunicacional condiciona de manera directa el éxito o fracaso del producto final. El presente informe aborda el análisis y resolución de dos problemáticas de alto impacto en la ingeniería de software contemporánea, estructuradas bajo el marco conceptual de la Unidad VI: "Liderazgo, Equipos y Cultura de Ingeniería". En la primera pregunta, se examina el impacto de la Ley de Conway en una entidad bancaria con arquitectura monolítica y silos funcionales, proponiendo un rediseño organizacional fundamentado en la Maniobra Inversa de Conway y las cuatro topologías de equipos del modelo Team Topologies (Stream-aligned, Platform, Complicated-subsystem y Enabling teams) para habilitar una migración efectiva hacia microservicios. En la segunda pregunta, se aborda la dimensión humana y cultural del equipo técnico, analizando cómo la erosión de la Seguridad Psicológica en los procesos de revisión de código (Pull Requests) degrada la velocidad de entrega, inhibe la innovación y destruye la motivación intrínseca. A partir de este diagnóstico, se formula una estrategia de intervención sustentada en el Liderazgo de Servicio, la Comunicación No Violenta (CNV) y dinámicas adaptativas de retrospectiva para reconstruir la confianza e institucionalizar una cultura DevOps orientada al aprendizaje continuo y al alto rendimiento.

Pregunta 1.

“La Maniobra Inversa de Conway y Team Topologies Contexto, una empresa bancaria tiene una arquitectura de software monolítica. Su estructura organizacional es funcional: un "Departamento de Base de Datos", un "Departamento de Backend Java" y un "Departamento de UI". Cada vez que quieren lanzar una nueva funcionalidad de "Créditos", los tres departamentos deben coordinarse mediante un Gerente de Proyecto tradicional, generando demoras de meses y constantes conflictos. Analice el problema actual utilizando la Ley de Conway. ¿Por qué la estructura organizacional está garantizando el fracaso de la agilidad? Proponga un rediseño organizacional utilizando los 4 tipos de equipos de Team Topologies (Streamaligned, Platform, Enabling, Complicated-subsystem). ¿Cómo quedarían conformados los equipos para soportar una futura migración a microservicios? ¿Qué rol jugaría el antiguo "Gerente de Proyecto" en esta nueva estructura organizacional?”

1. Análisis del problema actual mediante la Ley de Conway.

La empresa bancaria tiene una estructura organizacional muy rígida. Esta rigidez afecta directamente la forma en que se diseña su arquitectura tecnológica. Según la Ley de Conway, los sistemas que se crean reflejan la manera en que se comunican las personas dentro de una organización. Esta ley fue creada por Conway en 1968. En este caso, la estructura de la empresa se puede ver claramente en la arquitectura tecnológica que tiene. La manera en que la organización está dividida y cómo se comunican sus departamentos influye en cómo se construyen los sistemas informáticos. Esto genera problemas que se pueden observar fácilmente en la actual arquitectura tecnológica de la empresa:

- **Los departamentos están separados por especialidades técnicas:** hay un “Departamento de Base de Datos”, un “Departamento de Backend Java” y un “Departamento de UI”. Esta separación crea barreras entre los equipos. El sistema tiene que funcionar como un monolito, muy unido entre sus partes. Ningún departamento puede hacer cambios por su cuenta. Por ejemplo, si se cambia algo relacionado con “Créditos”, se tienen que actualizar al mismo tiempo la interfaz, la lógica de negocio y el esquema de la base de datos.
- **Transferencia constante de contexto (Handoffs) y latencia:** Cada requerimiento pasa de un departamento a otro con aprobaciones y colas de trabajo. Estas transferencias de contexto hacen que los tiempos de espera sean altos, el lead time desproporcionado aumente, se pierda información técnica y la responsabilidad sobre el resultado final en producción se diluya.
- **El gerente de proyecto actúa como un cuello de botella:** Sin comunicación directa entre las diferentes áreas, se necesita a un gerente de proyecto tradicional para coordinar los tres departamentos de manera burocrática. Esta forma de coordinación manual intenta lograr agilidad en una estructura que en realidad no la acepta. Esto genera conflictos entre los departamentos y retrasos que pueden durar varios meses.

1.2 ¿Por qué la estructura organizacional está garantizando el fracaso de la agilidad?

La estructura actual de la organización está condenada al fracaso en la agilidad. Aunque se usan prácticas ágiles como cadencias y iteraciones, la forma en que se comunican las personas sigue siendo centralizada y muy fragmentada. Esto no funciona. Según la Ley de Conway, si la organización está dividida en silos, el software que produce también será así. Un sistema

modular, como los microservicios, no puede surgir de una estructura que no coopera entre departamentos. La arquitectura del software refleja la forma en que trabaja la organización. Si no cambia la organización, no cambiará el software.

1.3 Rediseño Organizacional: Conformación de los Stream-Aligned Teams

Para prepararnos para la migración a microservicios en el futuro y reducir los problemas operativos, usamos la Maniobra Inversa de Conway. Esta estrategia reorganiza los equipos de forma proactiva. La idea es crear la arquitectura de software que queremos. Se alinean las fronteras de los equipos con los contextos delimitados del negocio. Cada equipo se enfoca en un área específica del negocio. Esto ayuda a que el software sea más claro y funcional. La organización se adapta a las necesidades del software. La estructura del equipo refleja la estructura del sistema. Esto mejora la eficiencia y la claridad. Cada parte del sistema tiene su propia responsabilidad. No hay confusión entre las áreas. El resultado es un sistema más fácil de mantener y escalar.

Conformación del Stream-Aligned Team ("Créditos")

Se rompen las líneas que separan los departamentos para formar un equipo que trabaja de manera integrada. Este equipo se enfoca constantemente en el flujo de valor del área de Créditos:

STREAM-ALIGNED TEAM: CRÉDITOS	
Cantidad	Responsabilidad End-to-End
1	Product Owner (Priorización y visión del negocio de créditos)
1	Tech Lead / Arquitecto de Software

STREAM-ALIGNED TEAM: CRÉDITOS	
2-3	Desarrolladores Full-Stack / Backend (Java, Kotlin o Go)
1	Desarrollador Frontend (Especialista en UI/UX)
1	Especialista en Base de Datos / Data Engineer
1	Ingeniero de Calidad / QA Automation

Características operativas de este equipo:

- 1. Autonomía End-to-End (You build it, you run it):** El equipo tiene todas las habilidades para diseñar, crear, probar, lanzar y vigilar el microservicio de créditos en producción sin que otros departamentos tengan que aprobar.
- 2. La carga cognitiva se reduce:** El equipo no tiene que preocuparse por gestionar infraestructura ni motores de cifrado complejos. Se concentra solo en la lógica del negocio de créditos y en la experiencia del cliente.
- 3. Soporte a microservicios:** Al definir los límites del equipo alrededor del dominio de créditos, la interfaz de comunicación del equipo se convierte directamente en la API REST o gRPC del microservicio de créditos. Esto garantiza que el microservicio se pueda desplegar por sí mismo. También asegura que tenga un bajo acoplamiento con otros módulos bancarios. El enfoque en créditos ayuda a mantener la claridad y la separación entre las diferentes partes del sistema.

1.4 ¿Cómo quedarían conformados los equipos para soportar una futura migración a microservicios?

Para complementar al Stream-Aligned Team de "Créditos" y garantizar un flujo continuo de valor sin sobrecargar cognitivamente a los desarrolladores con complejidades infraestructurales o algorítmicas, se estructuran los otros tres tipos de equipos definidos por Skelton y Pais (2019) en el marco de Team Topologies:

A. Platform Team (Equipo de Plataforma)

- **Conformación:** Ingenieros de Infraestructura, especialistas en Cloud, SREs (Site Reliability Engineers) y expertos en CI/CD.
- **Misión y Responsabilidad:** Diseñar y soportar una Plataforma de Autoservicio (Self-Service Platform) tratada como un producto interno (X-as-a-Service).
- **Aporte a la Migración a Microservicios:** Abstrae la complejidad de la infraestructura underlying. Provee pipelines estandarizados de despliegue continuo, orquestación de contenedores (Kubernetes), gestión de mallas de servicios (Service Mesh) y sistemas centralizados de observabilidad (métricas DORA, logs y trazabilidad distribuida). Esto permite que el equipo de Créditos despliegue microservicios de manera autónoma.

B. Complicated-Subsystem Team (Equipo de Subsistema Complicado)

- **Conformación:** Especialistas en algoritmia avanzada, ingenieros de datos y expertos en seguridad criptográfica bancaria.
- **Misión y Responsabilidad:** Asumir dominios que demandan una altísima especialización matemática o técnica pesada, reduciendo la carga cognitiva de los equipos orientados al flujo de valor.

- **Aporte a la Migración a Microservicios:** Desarrolla y mantiene como un componente aislado el "Motor de Scoring Crediticio Predictivo y Detección de Fraude en Tiempo Real". Este subsistema expone APIs internas (REST/gRPC) estrictamente definidas, permitiendo que el microservicio de Créditos consuma el cálculo financiero como un servicio externo sin requerir lógica matemática compleja en su codebase.

C. Enabling Team (Equipo Facilitador / Capacitador)

- **Conformación:** Arquitectos de Software Sénior, Agile Coaches y consultores internos de calidad y seguridad DevOps.
- **Misión y Responsabilidad:** Actuar como consultores técnicos temporales para cerrar brechas de capacidades y transferir conocimiento estratégico sin asumir la ejecución del trabajo diario.
- **Aporte a la Migración a Microservicios:** No escriben código de producción para los sprints de Créditos. Su rol es capacitar a los miembros del Stream-Aligned Team en patrones de descomposición de monolitos (Strangler Fig Pattern), principios de Diseño Guiado por el Dominio (DDD), automatización de pruebas de integración y redacción de Registros de Decisiones de Arquitectura (ADRs). Una vez transferida la capacidad, el equipo facilitador se retira a apoyar otras áreas.

En el Modelo Operativo basado en Productos (Product Operating Model) y bajo la Maniobra Inversa de Conway, el rol tradicional del Gerente de Proyecto (basado en comando, control, seguimiento rígido de diagramas de Gantt y asignación manual de tareas) se vuelve obsoleto y contraproducente, ya que reintroduce cuellos de botella burocráticos.

1.5 ¿Qué rol jugaría el antiguo "Gerente de Proyecto" en esta nueva estructura organizacional?

El antiguo Gerente de Proyecto realiza una transición funcional hacia dos roles estratégicos dentro de la nueva topología:

1. Agile Coach / Delivery Lead (Enfoque en Procesos, Personas y Flujo):

- Asume la filosofía del Liderazgo de Servicio (Servant Leadership).
- En lugar de mandar o pedir reportes de estado, se enfoca en optimizar la cadencia de entrega, eliminar impedimentos organizacionales inter-equipos, gestionar dependencias externas y proteger la autonomía del Stream-Aligned Team ante interferencias de la alta gerencia.
- Promueve el uso de Radiadores de Información y métricas de flujo en lugar de informes manuales.

2. Product Owner / Product Manager (Enfoque en Negocio y Valor):

- Si el profesional posee un perfil orientado al dominio bancario y al mercado, asume la responsabilidad del producto de créditos.
- Es el encargado de definir la visión del producto, maximizar el retorno de inversión y priorizar el Backlog de trabajo basándose en el valor entregado al cliente final, trabajando mano a mano con el Tech Lead y los desarrolladores sin intervenir en "cómo" se implementa técnicamente la solución.

Pregunta 2.

“Usted es el nuevo Agile Coach / Líder Técnico de un equipo. Nota que en las revisiones de código (Pull Requests), el Arquitecto Principal (un desarrollador muy senior y brillante) utiliza comentarios sarcásticos y destructivos hacia los desarrolladores junior, tales como ¿En serio vas

a usar este patrón? Es de principiantes. Los juniors han dejado de proponer ideas en las retrospectivas y la velocidad de entrega ha caído. Diagnostique la situación utilizando el concepto de Seguridad Psicológica de Amy Edmondson. ¿Qué impacto tiene esto en el rendimiento del equipo a largo plazo? Diseñe una estrategia de intervención utilizando principios de Liderazgo de Servicio y Comunicación No Violenta. ¿Cómo abordaría el conflicto con el Arquitecto Principal en privado y cómo reestructuraría el proceso de revisión de código para objetivar el feedback? Proponga una dinámica para la próxima Retrospectiva que ayude a reconstruir la confianza del equipo.”

2.1 Diagnóstico de la situación de revisión de código (Pull Requests)

El análisis de las dinámicas actuales del equipo de ingeniería revela una erosión crítica de la Seguridad Psicológica, concepto fundamental desarrollado por Amy Edmondson (1999). Se define este estado como la creencia compartida de que el entorno es seguro para asumir riesgos interpersonales sin temor a represalias.

En este contexto, las intervenciones del Arquitecto Principal durante las revisiones de código han transformado una herramienta de calidad técnica en un mecanismo de exclusión.

Factores Críticos del Diagnóstico

- **Hostilidad en Canales Asíncronos:** El proceso de *Code Review*, que debería ser pedagógico, se ha distorsionado hacia la intimidación. Comentarios descalificadores como "*¿En serio vas a usar este patrón? Es de principiantes*" generan una cultura de vergüenza que bloquea la comunicación abierta.
- **Asimetría de Poder:** La jerarquía del Arquitecto Principal amplifica el impacto de sus críticas. La descalificación no se procesa como una observación técnica, sino como un

juicio sobre la validez profesional del desarrollador, silenciando cualquier debate constructivo.

- **Inhibición de Riesgos:** Siguiendo las conclusiones del *Project Aristotle* de Google, la falta de seguridad psicológica es el principal predictor de bajo rendimiento. El miedo a la humillación técnica obliga a los desarrolladores a adoptar posturas defensivas, ocultando dudas y evitando la innovación.

2.2 Consecuencias a largo plazo en el rendimiento del equipo

La ausencia sostenida de un entorno seguro trasciende el malestar individual y genera un impacto directo en la resiliencia operativa y la calidad del producto final.

1. Degradación de la Velocidad de Entrega (Throughput)

La inseguridad psicológica provoca una parálisis por análisis. Los desarrolladores, temiendo el escrutinio punitivo, retrasan la apertura de *Pull Requests* para sobrediseñar soluciones defensivas. Este aislamiento incrementa el *Lead Time*, ya que las dudas no se consultan a tiempo para evitar parecer incompetentes.

2. Inoperancia de los Rituales Ágiles

Las retrospectivas y ceremonias de equipo pierden su valor estratégico. Se instaura una Cultura del Silencio (*Silence Culture*) donde los problemas latentes, como la deuda técnica o los fallos de proceso, no se mencionan por temor a reacciones hostiles, impidiendo la mejora continua.

3. Fuga de Talento y Atrición

El desgaste emocional y la falta de autonomía, maestría y propósito (pilares de la motivación según Daniel Pink) conducen inevitablemente al *burnout*. La pérdida de capital

intelectual por la rotación de desarrolladores junior y middle genera sobrecostos masivos en reclutamiento y una desestabilización del conocimiento del negocio.

4. Estancamiento Tecnológico

La innovación requiere un margen seguro para la experimentación. Bajo un liderazgo hostil, el equipo adopta una postura conservadora de "mínimo esfuerzo seguro", alineándose estrictamente con los sesgos del Arquitecto para evitar conflictos. Esto petrifica la arquitectura y limita el crecimiento técnico global de la organización.

2.3 Intervención, Reestructuración de la Revisión de Código y Retrospectiva

La solución tiene tres acciones que se aplican en orden. La primera es una intervención privada con el Arquitecto Principal, basada en el Liderazgo de Servicio y la Comunicación No Violenta. La segunda es una reestructuración del proceso de revisión de código que vuelve objetivo el feedback. La tercera es una dinámica de retrospectiva pensada para recuperar la confianza del equipo. El orden importa. La conversación privada protege la dignidad de las personas. El cambio de proceso evita que la mejora dependa de la buena voluntad de una sola persona. La retrospectiva devuelve la voz al equipo.

2.4 Estrategia de Intervención con el Arquitecto Principal

Fundamento: Liderazgo de Servicio

En la ingeniería de software moderna, el liderazgo dejó atrás el modelo de comando y control. En el Liderazgo de Servicio, el líder no asigna tareas ni castiga. Elimina impedimentos, protege al equipo y crea las condiciones para que se autoorganice. Desde este enfoque, el impedimento no es el Arquitecto como persona. El impedimento es la forma en que entrega su

feedback. El Arquitecto es el miembro con más conocimiento técnico del equipo. Apartarlo tendría un costo alto y además enviaría un mensaje de miedo, que es justo lo que se busca eliminar. Por eso el objetivo de la intervención es convertirlo de un cuello de botella en un multiplicador del talento del equipo, es decir, en un mentor.

La situación también es un *desafío adaptativo* en el sentido de Heifetz et al. (2009). No se resuelve con conocimiento técnico. Exige que una persona cambie su conducta y que el equipo aprenda una nueva forma de relacionarse. Por eso la intervención no puede limitarse a una orden. Debe generar comprensión y compromiso.

Herramienta: Comunicación No Violenta

La Comunicación No Violenta (CNV) permite expresar un problema sin atacar ni juzgar a la persona. Se organiza en cuatro componentes: observación, sentimiento, necesidad y petición. La Tabla muestra cómo se aplica cada uno a esta conversación.

Tabla

Aplicación de los Cuatro Componentes de la Comunicación No Violenta a la Conversación con el Arquitecto Principal

Componente	Qué implica	Frase propuesta
Observación	Hechos verificables, sin juicios ni etiquetas (no “eres sarcástico”).	“En los últimos cinco Pull Requests de los juniors hubo comentarios como este” .
Sentimiento	Emoción propia del líder, sin culpar al otro.	“Me preocupa, porque noto que los juniors ya no proponen ideas en las retrospectivas.”

Necesidad	El valor o la condición que está en juego.	“Necesito que el equipo se atreva a proponer y a equivocarse, porque sin eso no aprende ni entrega.”
Petición	Acción concreta, positiva y negociable; no una orden.	“¿Estarías dispuesto a que tus comentarios expliquen el porqué técnico y propongan una alternativa, y a ser mentor de dos juniors este sprint?”

Desarrollo de la Conversación Privada

La conversación se hace en privado, sin testigos y fuera de los canales del equipo. Una corrección pública pondría al Arquitecto a la defensiva y repetiría la misma humillación que se quiere eliminar. La conversación sigue estos pasos:

1. *Preparación.* El líder reúne ejemplos textuales de los comentarios desde el historial del repositorio y revisa sus propios supuestos para llegar sin juicios.
2. *Apertura con reconocimiento.* Empieza valorando la contribución técnica del Arquitecto (“Tu criterio es de lo mejor que tenemos, y por eso quiero hablar contigo”). Así la conversación se presenta como una alianza y no como un juicio.
3. *Aplicación de la CNV.* El líder expone la observación, su sentimiento, la necesidad y la petición, en ese orden y según la Tabla .
4. *Escucha empática.* El líder pregunta qué necesita el Arquitecto. Es posible que su frustración venga de recibir código de baja calidad sin tener tiempo para enseñar. Esa

necesidad es legítima y se atiende con la reestructuración del proceso que se presenta en la siguiente sección.

5. *Acuerdo y seguimiento.* Ambos cierran compromisos concretos y fijan una revisión en dos semanas. Si la conducta no cambia, se escala de forma gradual: primero una segunda conversación y después la participación de recursos humanos o de la gerencia.

2.5 Reestructuración del Proceso de Revisión de Código

La conversación privada es necesaria, pero no suficiente. Si la calidad del feedback depende de la buena voluntad de una persona, el problema puede volver. Por eso la segunda acción cambia el proceso para que el feedback sea objetivo, predecible e igual para todos. La reestructuración tiene tres componentes.

Estandarización Mediante Linter e Integración Continua

Todo lo que se puede medir con una regla lo verifica una máquina. Un formateador automático, un linter con las reglas de estilo del equipo y un análisis estático de complejidad se ejecutan en el pipeline de integración continua de cada Pull Request. Así desaparecen de la revisión humana las discusiones sobre espacios, comillas o nombres, que son las más propensas a juicios de gusto. Además, el linter es neutral: si falla, falla igual para el junior y para el Arquitecto. La revisión humana queda para lo que de verdad necesita criterio experto, como el diseño, la arquitectura y la lógica de negocio.

Guía de Revisión Objetiva

El equipo redacta y aprueba una guía de revisión con tres reglas. Primera: se comenta el código, nunca a la persona. Segunda: todo comentario explica el porqué y propone una

alternativa. Tercera: todo comentario lleva una etiqueta que declara su intención, según el formato *Conventional Comments*. Las etiquetas son *pregunta*, *sugerencia*, *problema*, *detalle* y *elogio*, e indican si el comentario bloquea o no la fusión. La Tabla 2 muestra el efecto de la guía sobre el comentario que originó el conflicto.

Reescritura de un Comentario de Revisión Aplicando la Guía Objetiva

Versión	Comentario	Efecto en el autor
Antes	“¿En serio vas a usar este patrón? Es de principiantes.”	Se siente atacado, no sabe qué cambiar y deja de proponer.
Después	“ <i>sugerencia (no bloqueante)</i> : este Singleton puede dificultar las pruebas unitarias porque introduce estado global. ¿Qué opinas de usar inyección de dependencias? Te dejo el ADR-012, donde lo decidimos.”	Entiende el problema técnico, conoce una alternativa y aprende de una decisión documentada.

Enfoque Pedagógico

La revisión de código se usa también como un espacio de aprendizaje. El Arquitecto hace sesiones semanales de revisión en pareja con los desarrolladores junior, en vivo, donde explica su razonamiento. Las decisiones de diseño que se repiten se registran como Architecture Decision Records (ADR). Así pasan de ser conocimiento tácito a conocimiento explícito y trazable, y no se vuelven a discutir en cada Pull Request. Los revisores rotan para distribuir el conocimiento, y los Pull Requests se mantienen pequeños para que se revisen rápido. Con estas

prácticas, el Arquitecto deja de ser un juez y se vuelve un maestro. Esto también responde a la necesidad de maestría que Pink identifica como motor de la motivación intrínseca.

2.6 Dinámica para la Próxima Retrospectiva

Los desarrolladores junior dejaron de hablar precisamente en las retrospectivas, así que ahí debe empezar la recuperación de la confianza. La dinámica dura 60 minutos y sigue las cinco fases propuestas por Derby y Larsen (2006): abrir el espacio, recoger datos, generar ideas, decidir acciones y cerrar. Combina tres técnicas: el *Safety Check*, *Hero*, *Highlight*, *Hardship* y *Kudos*. La Tabla A resume la agenda.

Tabla A

Agenda de la Retrospectiva Orientada a Recuperar la Confianza

Fase (tiempo)	Actividad	Objetivo
1. Abrir el espacio (5 min)	Lectura de la Directiva Primaria (Kerth, 2001): todos hicieron el mejor trabajo posible con lo que sabían en ese momento.	Dejar claro que no se buscan culpables.
2. Safety Check (5 min)	Voto anónimo de 1 (“no diré nada”) a 5 (“puedo decir cualquier cosa”).	Medir la seguridad percibida y adaptar el formato.
3. Recoger datos (20 min)	<i>Hero</i> , <i>Highlight</i> , <i>Hardship</i> : cada persona escribe quién la ayudó, su mejor	Empezar por lo positivo y sacar a la luz las

	momento y su mayor dificultad del sprint.	dificultades sin exponer a nadie.
4. Decidir acciones (20 min)	El equipo redacta y vota su acuerdo de trabajo para las revisiones de código.	Que las normas sean del equipo y no impuestas.
5. Cerrar (10 min)	<i>Kudos</i> : tarjetas donde cada persona reconoce algo concreto de otra.	Terminar con reconocimiento y emociones positivas.

Reglas de Facilitación

El facilitador es el Agile Coach, no el Arquitecto, para mantener la neutralidad. Si el promedio del Safety Check es igual o menor a 2, toda la sesión se hace con notas anónimas. El formato se adapta al nivel de miedo del equipo, y no al revés. En la fase de recoger datos, las dificultades se escriben de forma individual antes de compartirlas, para que nadie tenga que exponerse primero. Si se acordó en la conversación privada, el Arquitecto abre la fase de decidir acciones reconociendo que el tono de sus revisiones no ayudó al equipo. Cuando el integrante con más estatus muestra vulnerabilidad, los demás perciben que es seguro hablar.

Indicadores de Seguimiento

Una sola sesión no recupera la confianza. La confianza se reconstruye con conductas que se repiten en el tiempo. Por eso se miden tres indicadores en las siguientes retrospectivas: la evolución del promedio del Safety Check, la cantidad de propuestas hechas por desarrolladores junior y el tiempo que tarda un Pull Request desde que se abre hasta que se fusiona. Si mejoran,

la intervención funciona. Si se estancan, el equipo vuelve a revisar la estrategia en la retrospectiva siguiente.

CONCLUSIONES

La Ley de Conway y la Maniobra Inversa como Determinantes Arquitectónicos: Se demostró que la arquitectura de software no es un ente aislado de la realidad social de la empresa. Las estructuras organizacionales tradicionales divididas en silos funcionales imponen inevitablemente acoplamiento rígido y sistemas monolíticos. La adopción de la Maniobra Inversa de Conway, mediante el modelo Team Topologies, permite diseñar intencionalmente la estructura de los equipos alrededor de contextos delimitados de negocio (Bounded Contexts), sentando las bases organizacionales indispensables para sostener una arquitectura descentralizada de microservicios con alta autonomía y bajo acoplamiento.

Optimización del Flujo mediante la Reducción de la Carga Cognitiva: La clasificación estratégica en equipos Stream-aligned, Platform, Complicated-subsystem y Enabling garantiza que los equipos orientados al negocio no sufran sobrecarga cognitiva. La provisión de servicios de plataforma autoservicio y el soporte de subsistemas especializados permiten acelerar el tiempo de ciclo (Lead Time), eliminar transferencias burocráticas de contexto (handoffs) y cumplir con la premisa operativa DevOps: "You build it, you run it".

La Seguridad Psicológica como Predictor N° 1 del Alto Rendimiento: El diagnóstico de las dinámicas de equipo evidenció que la excelencia técnica carece de impacto positivo si se ejerce bajo esquemas de liderazgo hostiles. La erosión de la seguridad psicológica en actividades clave como los Code Reviews desmotiva al personal, bloquea la innovación y destruye los espacios de mejora continua. La implementación del Liderazgo de Servicio, la Comunicación No Violenta (CNV) y las retrospectivas orientadas a la confianza son condiciones sine qua non para cultivar perfiles integrales (en forma de "T") y sostener una cultura de ingeniería saludable y de alto rendimiento.

REFERENCIAS

- Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4.^a ed.). Addison-Wesley.
- Cockburn, A. (2006). Agile software development: The cooperative game (2.^a ed.). Addison-Wesley.
- Conway, M. E. (1968). How do committees invent? *Datamation*, 14(4), 28–31.
- Edmondson, A. (1999). Psychological safety and learning behavior in work teams. *Administrative Science Quarterly*, 44(2), 350–383.
- Forsgren, N., Humble, J., & Kim, G. (2018). Accelerate: The science of lean software and DevOps: Building and scaling high performing technology organizations. IT Revolution Press.
- Greenleaf, R. K. (1977). Servant leadership: A journey into the nature of legitimate power and greatness. Paulist Press.
- Heifetz, R. A., Grashow, A., & Linsky, M. (2009). The practice of adaptive leadership: Tools and tactics for changing your organization and the world. Harvard Business Press.
- Humble, J., & Farley, D. (2010). Continuous delivery: Reliable software releases through build, test, and deployment automation. Addison-Wesley.
- IEEE Computer Society. (2014). Guide to the software engineering body of knowledge (SWEBOK V3/V4). IEEE Computer Society.
- Newport, C. (2016). Deep work: Rules for focused success in a distracted world. Grand Central Publishing.
- Nygard, M. (2011). Documenting architecture decisions. Cognitect Blog.
- Pink, D. H. (2009). Drive: The surprising truth about what motivates us. Riverhead Books.